Architectural Security Analysis of Electro-Mechanical Braking Systems and AI-Based Threat Vectors

*Note: Sub-titles are not captured in Xplore and should not be used

1st Given Name Surname
dept. name of organization (of Aff.)
name of organization (of Aff.)
City, Country
email address or ORCID

2nd Given Name Surname
dept. name of organization (of Aff.)
name of organization (of Aff.)
City, Country
email address or ORCID

3rd Given Name Surname
dept. name of organization (of Aff.)
name of organization (of Aff.)
City, Country
email address or ORCID

4th Given Name Surname
dept. name of organization (of Aff.)
name of organization (of Aff.)
City, Country
email address or ORCID

5th Given Name Surname
dept. name of organization (of Aff.)
name of organization (of Aff.)
City, Country
email address or ORCID

6th Given Name Surname
dept. name of organization (of Aff.)
name of organization (of Aff.)
City, Country
email address or ORCID

***Abstract*—Electro-mechanical brake (EMB) plays a key role in technology for future brake-by-wire vehicles, but introduces new cybersecurity threats because of their independence on electronic controllers, sensors, wheel-actuators, and communication networks. This research paper presents a Risk Assessment (TARA) and threat Analysis methodology for the STRIDE threat model and EMB systems based on ISO/SAE 21434. Three EMB architectures are MCU + Axle ECU, MCu + Pedal-direct, and MCu + Auxiliary ECU. Each architecture was separately modeled and analyzed using ANSYS Medini Analyze. Five AI models are analyzed sequentially with common prompt for idnetifies ability of STRIDE-based threats. The results of the threat analysis are compared with the STRIDE-category and the consistency of the rist assessment. The study shows the comparison between AI-assisted threat analysis and tools-suppported TARA for EMB architectures.**

***Index Terms*—Key words: Electro-Mechanical Brake (EMB); Brake-by-wire (BBW); Risk Assessment (TARA) and Threat Analysis; Automotive Cybersecurity; STRIDE; ISO?SAE 21434; ANSYS Medini Analyze; Artificial Intelligence.**

I. Introduction

A. Background: Electro-Mechanical Brake Systems and Cybersecurity

Modern vehicles are gradually moving from hydraulic and mechanical control toward brake-by-wire (BBW) architectures, in which the brake pedal is decoupled from the wheel-end actuators and braking commands are transmitted electrically. Among BBW systems, electro-mechanical brake (EMB) systems have received increasing attention due to their distributed architecture. As illustrated in Fig. 1, an EMB system replaces the hydraulic calipers of a conventional braking system with electro-mechanical wheel-end actuators (WEAs), each containing a servo motor, a transmission mechanism, a conversion mechanism, and a brake caliper. The brake pedal, sensors, main control unit (MCU), power supply, and communication bus form the remaining core components. Each WEA is controlled independently through electrical signals rather than hydraulic pressure, which enables per-wheel braking-force control and flexible chassis integration. This distributed design is relevant for high-level autonomous driving, where fast and accurate braking actuation is required.

However, the absence of mechanical backup in EMB systems introduces a safety and security concern. Any component—whether in the sensors, MCU, communication bus, or WEA—can be targeted by cyberattacks, potentially leading to loss of braking, unintended braking, or braking deviation. Unlike random hardware faults addressed by ISO 26262, these threats are intentional and call for a cybersecurity perspective. The ISO/SAE 21434 standard provides a framework for identifying and mitigating such threats through Threat Analysis and Risk Assessment (TARA).

TARA is a systematic methodology for identifying cybersecurity threats, assessing their impact, and evaluating attack feasibility [2]. As defined in ISO/SAE 21434, it consists of seven main steps: asset identification, damage scenario identification, impact rating, attack path analysis, attack feasibility rating, risk value calculation, and risk treatment decision. The output of TARA is a set of cybersecurity goals and requirements that guide the design of secure systems. To structure the threat identification process, the STRIDE threat

Identify applicable funding agency here. If none, delete this.

model is commonly employed. STRIDE classifies threats into six categories: Spoofing (impersonating an entity), Tampering (unauthorized modification), Repudiation (denying an action), Information Disclosure (exposing data), Denial of Service (degrading availability), and Elevation of Privilege (gaining unauthorized access). Each category corresponds to a violated security property (authenticity, integrity, accountability, confidentiality, availability, and authorization, respectively). The STRIDE framework is particularly useful for automotive systems because it maps naturally to the communication interfaces and control signals present in EMB architectures.

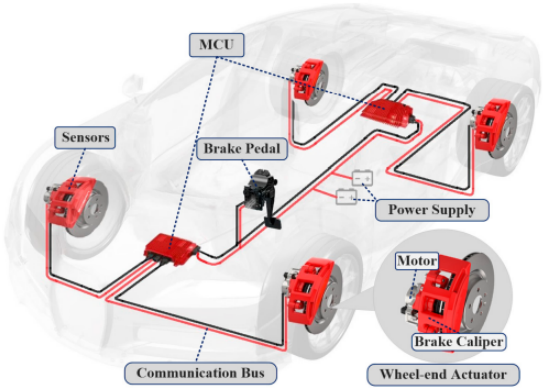


Fig. 1. Overall scheme of an electro-mechanical brake (EMB) system [2], showing the MCU, brake pedal, sensors, power supply, communication bus, and four wheel-end actuators (WEAs). Each WEA contains a servo motor, transmission and conversion mechanisms, and a brake caliper.

However, the absence of mechanical backup in EMB systems introduces a safety and security concern. Any component—whether in the sensors, MCU, communication bus, or WEA—can be targeted by cyberattacks, potentially leading to loss of braking, unintended braking, or braking deviation. Unlike random hardware faults addressed by ISO 26262, these threats are intentional and call for a cybersecurity perspective. The ISO/SAE 21434 standard provides a framework for identifying and mitigating such threats through Threat Analysis and Risk Assessment (TARA) and the STRIDE threat model.

B. Motivation and Contributions

While TARA has been applied to autonomous emergency braking (AEB) systems, existing studies focus on the AEB level rather than the EMB actuator level, which is the underlying safety-critical component. In addition, no existing study provides a comparative TARA across multiple candidate EMB architectures, nor does it examine how AI-assisted tools can complement expert-driven TARA in this domain. These observations motivate our work.

This paper addresses these gaps by proposing a TARA-based threat analysis methodology for EMB systems. The methodology evaluates three candidate EMB architectures—(a) MCU + Axle ECU, (b) MCU + Pedal-direct, and (c) MCU + Auxiliary ECU—using ANSYS Medini Analyze and AI-assisted tools. The main contributions are:

- A TARA framework aligned with ISO/SAE 21434 and the STRIDE threat model for EMB systems.
- A comparative security evaluation of three EMB architectures, identifying attack surfaces such as sensor spoofing, communication bus injection, and actuator manipulation.
- A comparative analysis of AI-assisted versus Medini-based TARA in terms of threat coverage, risk consistency, and expert correction effort.

II. Related Work

This section reviews prior work across three domains: (1) EMB system architecture and functional safety, (2) braking control and thermal reliability, and (3) AEB performance and security assessment. For each domain, we summarize key contributions and identify the gap that motivates our work.

A. EMB System Architecture and Functional Safety

The EMB architecture analyzed in this work is based on the design proposed by Peng et al. [2], whose functional safety analysis provides the foundation for our cybersecurity extension. Their architecture is built around dual MCUs, four wheel-end actuators (WEAs), dual power supplies, and a multi-channel communication backbone (CAN, LIN, FlexRay). Using ISO 26262, they derive three ASIL-D safety goals (avoiding insufficient deceleration, unintended deceleration, and excessive lateral displacement) and allocate technical safety requirements to each component. Their core technical contribution is a two-stage reliability analysis of the WEA matrix. First, fault tree analysis (FTA) identifies the minimal cut sets causing MCU and WEA failures and derives a single-WEA failure rate of 21.5 FIT. Second, since WEAs fail in random combinations, a Markov chain models six possible operating states of the WEA matrix (all healthy, one failed, two failed on the same side, two failed diagonally or on the same axle, three failed, all four failed). A fault-tolerant control (FTC) strategy redistributes braking force across the remaining healthy WEAs, and hardware-in-the-loop (HIL) testing shows that three of the six states (all-healthy, single-failure, diagonal/axle double-failure) still meet the safety goals, while the other three (same-side double, triple, total failure) do not and are classified as "unavailable" states. Combining the steady-state Markov probabilities of the unavailable states with the component failure-rate table, the authors report a WEA matrix unavailability of 9.244×10^{-3} FIT and PMHF values of 6.14, 5.89, and 6.03 FIT for the three safety goals, all below the 10 FIT ASIL-D threshold. Zhu and Li [3] extend the functional safety boundary by coupling EMB with steering, drive, and suspension systems, constructing a 4-way redundant architecture with a braking failure rate of $Q_0 \approx 2.0003 \times 10^{-10}$. Faisal et al. [4] design and implement an EMB system using a BLDC motor and cascade position controller with stiffness curve estimation, demonstrating the practical feasibility of EMB control.

While these works provide a foundation for functional safety (ISO 26262), their failure analysis covers only random hardware faults. No threat modeling, TARA, or reference to

ISO/SAE 21434 appears in their analyses. Table I summarizes key limitations of [2] that motivate our work.

TABLE I
KEY LIMITATIONS OF PENG ET AL. [2]

Category	Limitation
Calculation	Eq. (4) and the 6×6 transition matrix contain unexplained simplifications; the reported PMHF values cannot be reconciled with Eq. (8) by straightforward algebra.
Cybersecurity	No threat modeling, TARA, or reference to ISO/SAE 21434; ACU firmware integrity and message authentication are not discussed, despite the ACU being flagged as a single point of failure in [2].
Modeling assumption	The Markov chain assumes WEAs fail independently and randomly, which a coordinated attack on a shared bus, gateway, or power circuit would violate.

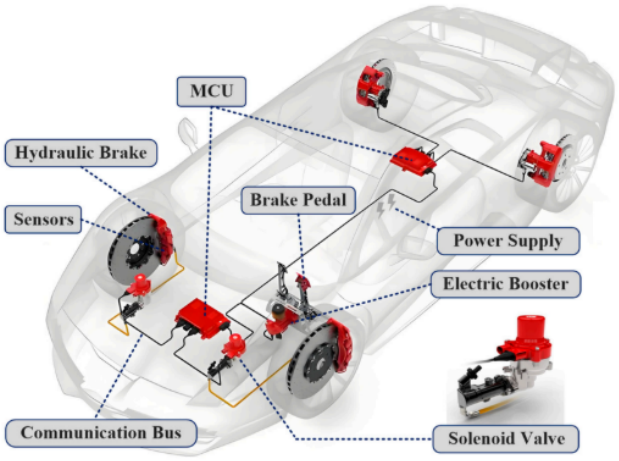


Fig. 2. Overall scheme of an electro-hydraulic brake (EHB) system [2], showing the main control unit (MCU), hydraulic brake, sensors, brake pedal, electric booster, solenoid valve, power supply, and communication bus.

B. Braking Control and Thermal Reliability

Beyond architectural design, prior work has addressed braking control algorithms and thermal reliability. Wang et al. [10] proposed a robust clamping-force trcaking strategy for EMB systems under nonlinearities, uncertainties, and disturbances. Their work demonstrates the importance of accurate actuation and robust control for achieving reliable braking behavior. Shi et al. [11] investigated brake-pressure estimation while considering the evolution of the brake-lining coefficient of friction. Their results highlight the importance of accurate estimation of braking-related variables under changing operating conditions. Gunjate and Khot [5] review ABS and EBS integration using PWM and fuzzy logic control, classifying ABS control techniques and noting that ABS–EBS integration contributes to reduced collision risk. Zhao et al. [6] provide a review of One-Box EHB systems, covering architecture design, master cylinder and wheel-cylinder pressure control, and electro-hydraulic composite braking, while noting that redundant braking systems are relevant for high-level autonomous driving. Park et al. [7] address the thermal management of EMB motors under extreme conditions (disc temperatures exceeding 700°C), proposing a hybrid passive cooling system that reduces the maximum coil temperature by 23.2 K and identifying SAT as the best-performing PCM among those tested.

These works contribute insights into control and thermal reliability, but they do not address safety and security analysis at the architectural level.

C. AEB Performance and Security Assessment

The performance of AEB systems has been studied in various scenarios. Ribeiro et al. [8] evaluate AEB across three highway scenarios (CCRm, CCRs, CCRb), observing that while AEB prevents collisions under controlled conditions, its effectiveness can degrade in complex traffic and sudden braking events. They also discuss integration challenges with Path Planning and Adaptive Cruise Control (ACC). Della Monica et al. [9] perform a TARA of an AEB system using ISO/SAE 21434 and the STRIDE threat model, identifying attack vectors and security vulnerabilities. Their work is among the first to apply TARA to AEB systems, but it focuses on the AEB level rather than the EMB actuator level, which is the underlying safety-critical component.

These works contribute to AEB performance and TARA, but a combined safety and security perspective at the EMB level has received limited attention, which is the direction our work pursues.

III. METHODOLOGY

This study proposes a comparative TARA methodology for EMB systems, in which the STRIDE-based threat analysis is performed both by a human analyst using ANSYS Medini Analyze and by five AI models, using the same prompt and the same architecture image for each architecture. The goal is to evaluate how different analysis tools perform across the STRIDE categories, which threats they identify or miss, and how their outputs compare in terms of coverage, hallucination, and architectural sensitivity.

The three candidate EMB architectures analyzed in this work are derived from the reference architecture proposed by Peng et al. [2]:

a) MCU + Pedal-direct.: This is the simplest and most cost-effective configuration. There is no intermediate ECU tier: a single MCU receives the brake pedal signal and communicates directly with all four WEAs over a shared bus. Power follows a full redundancy layout (two independent sources covering the system), while communication follows an "X"-circuit layout, which coordinates braking force distribution more effectively than an "H" circuit [2] at the cost of more complex wiring. As noted in [2], this architecture "provides poor residual capability after the MCU failure while reducing cost"—if the single MCU fails, there is no backup controller.

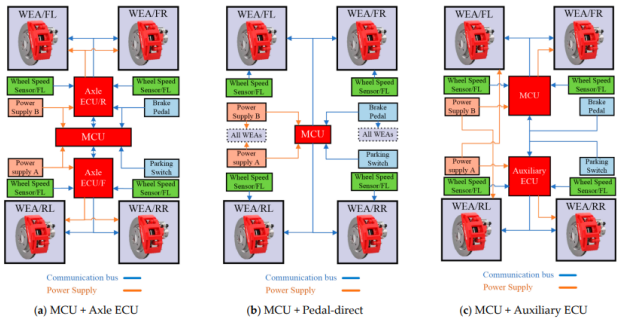


Fig. 3. Typical EMB system architectures: (a) MCU + Axle ECU, (b) MCU + Pedal-direct, (c) MCU + Auxiliary ECU. reprinted from [2]

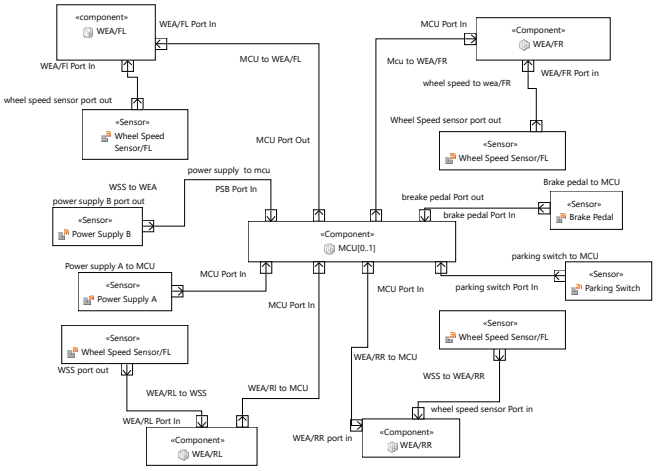


Fig. 4. MCU + Pedal-direct

b) MCU + Auxiliary ECU.: This architecture is an intermediate compromise. The primary MCU is supplemented by a single Auxiliary ECU for backup—unlike (a), this auxiliary controller is not dedicated to a specific axle but supports the overall system. Power follows an "X"-circuit layout (as in (b)), while communication follows an "H"-circuit layout (as in (a)). Consequently, it inherits the higher resilience of the "X" circuit for power delivery but retains the same-side vulnerability of the "H" circuit for data communications [2]. The reference study characterizes this as a pragmatic compromise, "balancing controller redundancy with cost efficiency."

c) MCU + Axle ECU: Braking control is distributed across a central MCU and two dedicated axle ECUs—one for the front axle (Axle ECU/F) and one for the rear axle (Axle ECU/R). Each Axle ECU sits between the MCU and the two WEAs of its axle, managing two wheels rather than all four. Power and communication follow an "H"-circuit layout, described in [2] as a same-side redundancy design: simpler to implement, but more vulnerable if an entire vehicle side loses its circuit. This configuration has the highest control redundancy among the three, at the cost of a higher component count.

Each architecture is analyzed independently, and the results

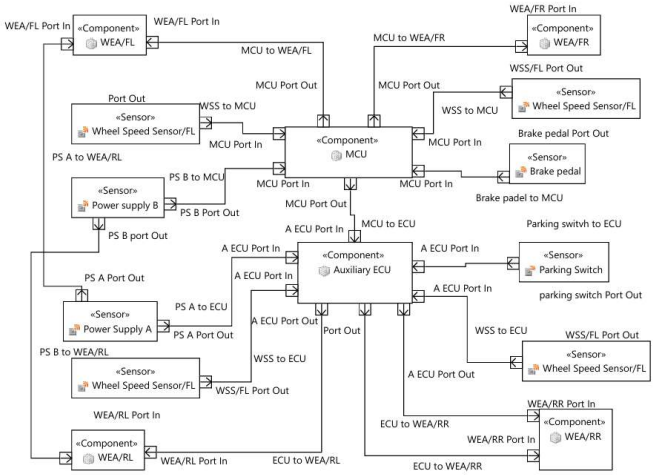


Fig. 5. MCU + Auxiliary ECU

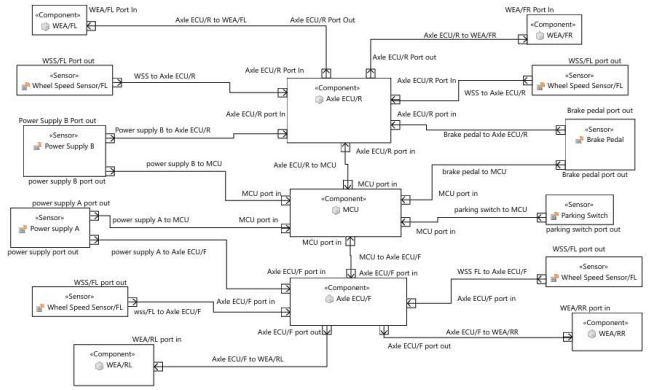


Fig. 6. MCU + Axle ECU

are compared across tools and architectures.

A. Model Summary

Table II summarizes the three candidate architectures.

B. Reference Analysis Using ANSYS Medini Analyze

ANSYS Medini Analyze is used as the reference platform for the STRIDE-based TARA. The workflow follows ISO/SAE 21434 and consists of the following steps:

1) **Asset Identification:** For each architecture, assets are identified, including brake pedal signals, wheel speed data, MCU control commands, WEA actuation signals, and communication bus messages.
2) **Trust Boundary Definition:** Trust boundaries are defined between the in-vehicle network (CAN, FlexRay, LIN) and external interfaces, as well as between sensors and the MCU.
3) **STRIDE-Based Threat Modeling:** For each asset and interface, STRIDE categories (Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service,

TABLE II
COMPARATIVE SUMMARY OF THE THREE EMB ARCHITECTURES

Feature	(a)	(b)	(c)
Controller	MCU + 2 Axle ECUs	Single MCU	MCU + Aux ECU
Pedal-to-WEA	Indirect	Direct	Indirect
Power Layout	"H"-circuit	Full red.	"X"-circuit
Comm. Layout	"H"-circuit	"X"-circuit	"H"-circuit
Redundancy	High (axle)	Low (none)	Medium (system)
Residual After MCU Failure	Good	Poor [2]	Intermediate
Cost	High	Low	Medium
Wiring Complexity	High	Low	Medium
Fault Isolation	Per-axle	None	Partial
Reference Summary	"Strongest redundancy, high cost"	"Poor residual, low cost"	"Balanced"

Elevation of Privilege) are applied to identify threat scenarios.

4) **Attack Path Analysis:** Attack paths are constructed for each threat scenario, and attack feasibility is assessed using ISO/SAE 21434 attack potential criteria.
5) **Risk Assessment:** Impact and feasibility are combined to assign risk levels (Critical, High, Medium, Low).
6) **Security Goal Derivation:** Cybersecurity goals and security requirements are derived for Critical and High-risk threats.
7) **Traceability:** All threats, risks, and requirements are traced across architecture, hardware, and software layers.

The Medini-based TARA serves as the reference against which the AI-generated TARAs are compared.

C. AI-Assisted STRIDE Analysis

Five AI models are evaluated in this study:

- Perplexity Sonar
- Gemini 3.8 Flash
- DeepSeek-V3
- Claude Sonnet 5
- GPT-5.6 Luna

Each model is given the same prompt and the same architecture image, following the template provided in Appendix A. The prompt instructs the model to identify STRIDE threats for the given architecture, using the component names exactly as labelled in the figure, and to fill in a structured table with the following columns: ID, Name, Assets, To Be Assessed, Category, and Severity. The prompt also instructs the model to output the total number of threats per STRIDE category.

The AI analyses are performed independently for each of the three architectures, resulting in 15 AI-generated TARA tables (5 models $\times$ 3 architectures).

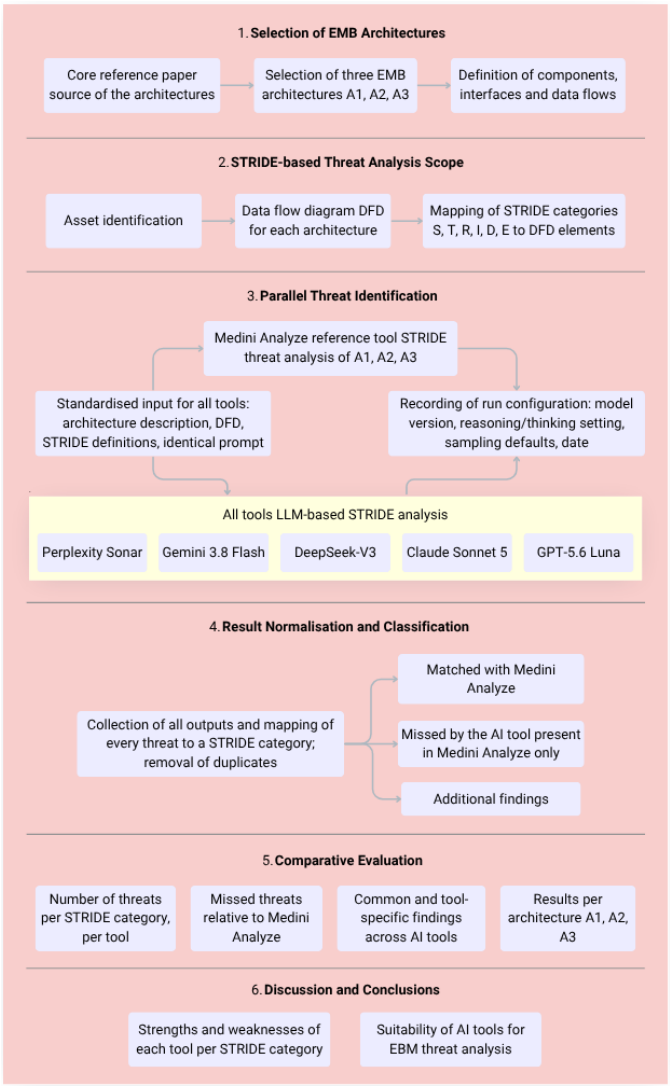


Fig. 7. Methodology workflow: (1) selection of EMB architectures from the core reference paper, (2) definition of the STRIDE-based threat analysis scope, (3) parallel threat identification using Medini Analyze and five AI models, (4) normalization and classification of results, (5) comparative evaluation across STRIDE categories and architectures, and (6) discussion and conclusions.

IV. RESULTS

This section presents the comparative results of the STRIDE-based threat analysis across the three EMB architectures, the five AI models, and the Medini reference. The results are organized around five aspects: (1) threat coverage per architecture, (2) STRIDE category distribution, (3) severity ratings, (4) threat detection comparison matrices, and (5) tool-specific findings.

A. Threat Coverage Across Architectures

Fig. 8 summarizes the detection performance of each AI model relative to the Medini reference across the three architectures. Recall values vary notably by architecture and by model. For MCU + Pedal-direct, DeepSeek achieves the highest recall (100%), followed by ChatGPT (69%), Claude (44%),

Gemini (42%), and Perplexity (33%). For MCU + Auxiliary ECU, the highest recall is observed for DeepSeek (75%), followed by ChatGPT (72%), Claude (69%), Gemini (44%), and Perplexity (33%). For MCU + Axle ECU, Perplexity shows the highest recall (89%), followed by Claude (69%), ChatGPT (58%), DeepSeek (53%), and Gemini (44%). This suggests that model performance is architecture-dependent rather than uniformly consistent.

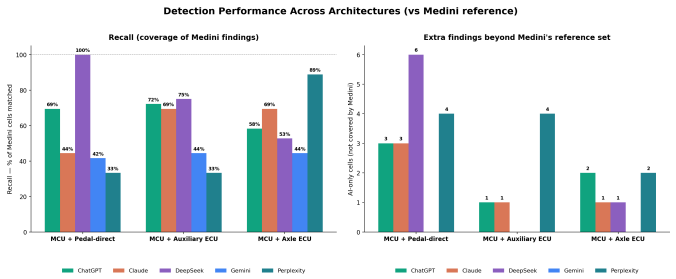


Fig. 8. Detection performance across architectures: recall and AI-only findings.

The right-hand panel of Fig. 8 shows the number of AI-only findings not present in the Medini reference set. DeepSeek produces the highest number of additional findings for MCU + Pedal-direct (6), while Perplexity produces 4 additional findings for both MCU + Auxiliary ECU and MCU + Axle ECU. These additional findings may represent either genuine gaps in the Medini analysis or hallucinated threats, and require expert validation.

The bar charts in Figs. 9, 11, and 10 show the total number of threats identified by each tool per architecture. Medini consistently reports the highest number of entries (243 for MCU + Axle ECU, 228 for MCU + Auxiliary ECU, and 192 for MCU + Pedal-direct), while Gemini reports the lowest counts (34, 29, and 24, respectively). Claude and Perplexity report intermediate counts, and DeepSeek and ChatGPT show similar totals across architectures.

B. STRIDE Category Distribution

Fig. 12 shows the STRIDE category coverage profile across all three architectures combined. Medini reports the highest number of entries in Tampering, Information Disclosure, and Denial of Service (178 each), followed by Spoofing (43), Repudiation (43), and Elevation of Privilege (43). Among the AI models, Claude reports the highest number of Tampering threats (78) and Denial of Service threats (80). ChatGPT and Perplexity report comparable counts across most categories, while Gemini consistently reports the lowest number of threats. Repudiation is the least covered category across all AI models, with Gemini reporting only 6 entries.

An interesting inversion is observed in the Spoofing category: Medini reports 43 Spoofing threats across the three architectures, while several AI models report more—Claude (57), ChatGPT (50), and Perplexity (50).

The per-architecture STRIDE comparison in Table 13 shows that the MCU + Axle ECU architecture consistently receives

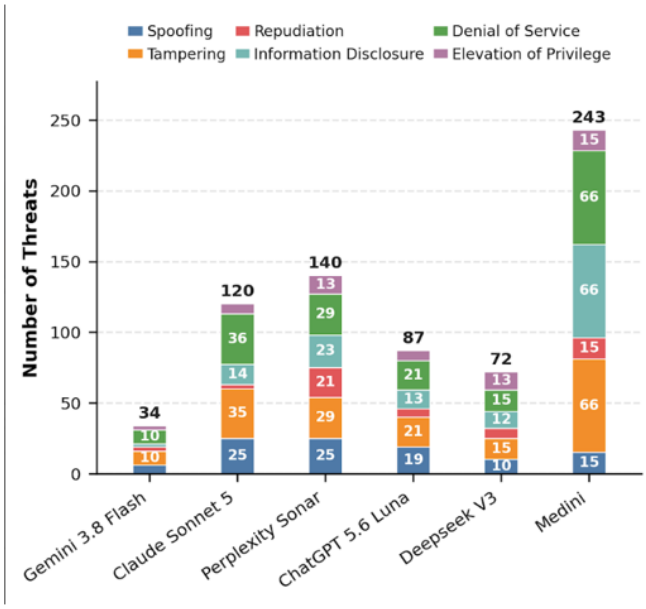


Fig. 9. Total number of threats per tool for MCU + Axle ECU.

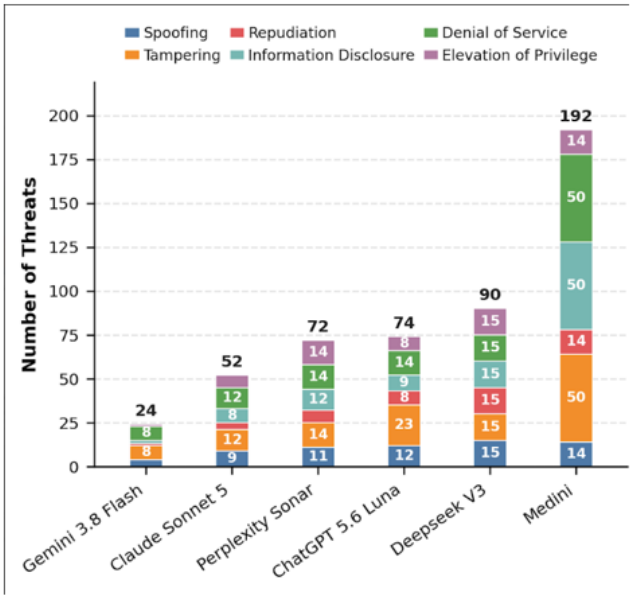


Fig. 10. Total number of threats per tool for MCU + Pedal-direct.

the highest number of threats in all six categories, followed by MCU + Auxiliary ECU and MCU + Pedal-direct. This ordering is consistent across the three architectures and suggests that the number of identified threats increases with the number of components and communication links.

C. Severity Ratings

Fig. 14 shows the average severity score and the share of severe-rated threats by tool across the three architectures. The average severity score ranges from approximately 2.9 to 3.8 across tools and architectures, with DeepSeek

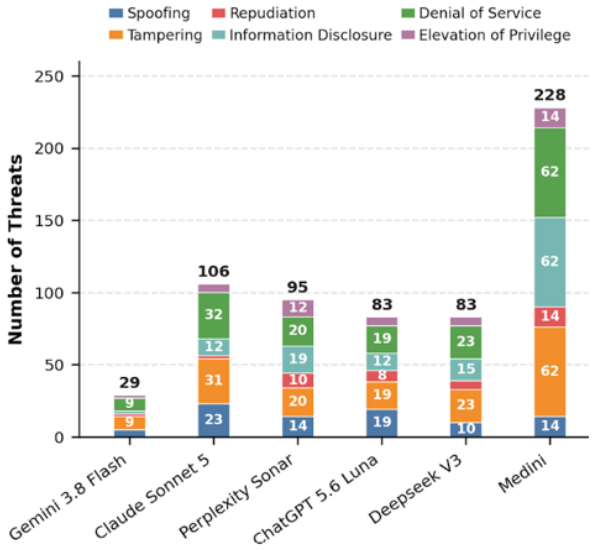


Fig. 11. Total number of threats per tool for MCU + Auxiliary ECU.

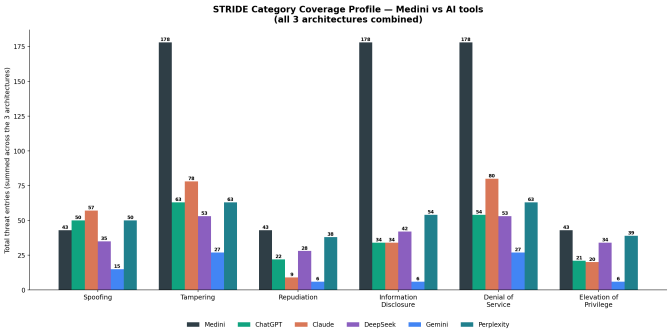


Fig. 12. STRIDE category coverage profile: Medini vs AI tools.

and Gemini showing the highest average severity scores for MCU + Auxiliary ECU and MCU + Axle ECU. The share of severe-rated threats is highest for Gemini (79–86%) and DeepSeek (54–75%), and lowest for Claude (38–64%).

The per-architecture severity distributions in Figs. 15, 17, and 16 show that the Communication Link and WEA asset groups receive the highest average severity scores across tools. Power Link and Wheel Speed Sensor receive more variable

Table 3. STRIDE-based threat comparison across architectures.

	MCU + Pedal-direct					MCU + Auxiliary ECU					MCU + Axle ECU				
STRIDE Category	**C**	**H**	**M**	**L**	**Total**	**C**	**H**	**M**	**L**	**Total**	**C**	**H**	**M**	**L**	**Total**
Spoofing	6	8	0	0	**14**	7	7	0	0	**14**	8	7	0	0	**15**
Tampering	29	17	4	0	**50**	37	25	0	0	**62**	41	25	0	0	**66**
Repudiation	0	0	0	14	**14**	0	0	0	14	**14**	0	0	0	15	**15**
Information Disclosure	0	0	0	50	**50**	0	0	0	62	**62**	0	0	0	66	**66**
Denial of Service	0	34	16	0	**50**	0	47	15	0	**62**	0	51	15	0	**66**
Elevation of Privilege	6	3	5	0	**14**	7	2	5	0	**14**	8	2	5	0	**15**

C = critical, H = high, M = medium, L = low. Total = sum of identified threats per category.

Fig. 13. STRIDE category coverage profile from Medini for three architecture

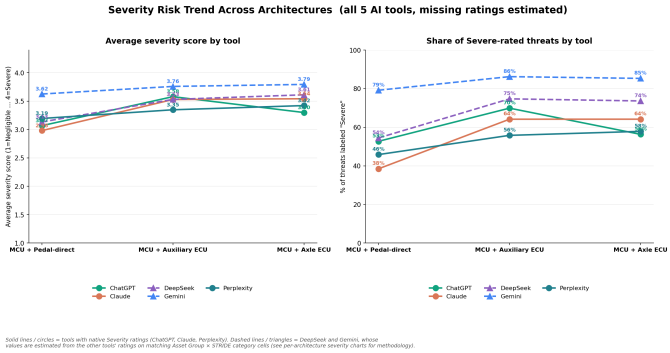


Fig. 14. Severity risk trend across architectures for all five AI tools.

ratings, with some tools assigning moderate or negligible severity where others assign major or severe.

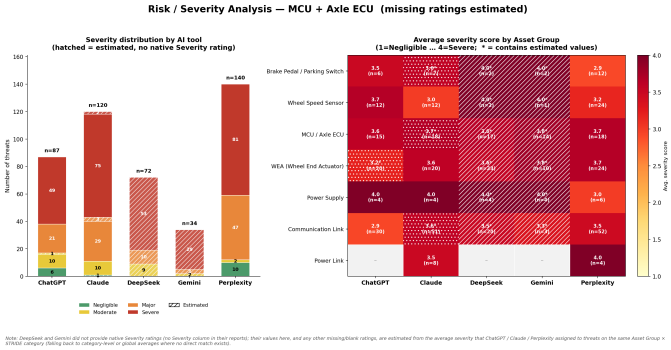


Fig. 15. Severity analysis for MCU + Axle ECU.

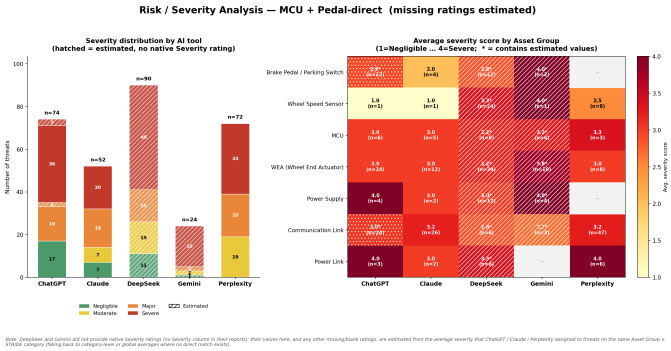


Fig. 16. Severity analysis for MCU + Pedal-direct.

It should be noted that DeepSeek and Gemini did not provide native severity ratings in their outputs. For these models, severity values were estimated from the ratings assigned by ChatGPT, Claude, and Perplexity to threats in the same asset group and STRIDE category. These estimated values are marked as hatched bars in the figures and should be interpreted with caution.

D. Threat Detection Comparison Matrices

Figs. 18, 20, and 19 present the threat detection comparison matrices for the three architectures. Each cell in the matrix

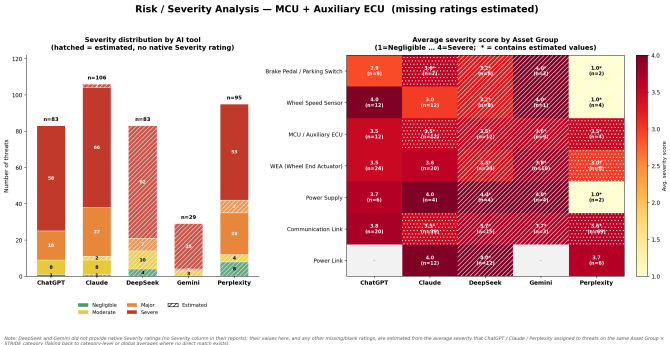


Fig. 17. Severity analysis for MCU + Auxiliary ECU.

represents an asset group and STRIDE category combination, and the values indicate the number of threats identified by each tool.

For MCU + Axle ECU, Medini reports 36 cells with at least one entry, while the AI models match between 16 and 32 of these cells. Perplexity matches 32 cells, ChatGPT matches 21, Claude matches 25, DeepSeek matches 19, and Gemini matches 16. Medini-only cells (threats identified by Medini but missed by the AI) range from 4 (Perplexity) to 20 (Gemini). AI-only cells (threats identified by the AI but not by Medini) range from 0 (Gemini) to 2 (ChatGPT, Perplexity).

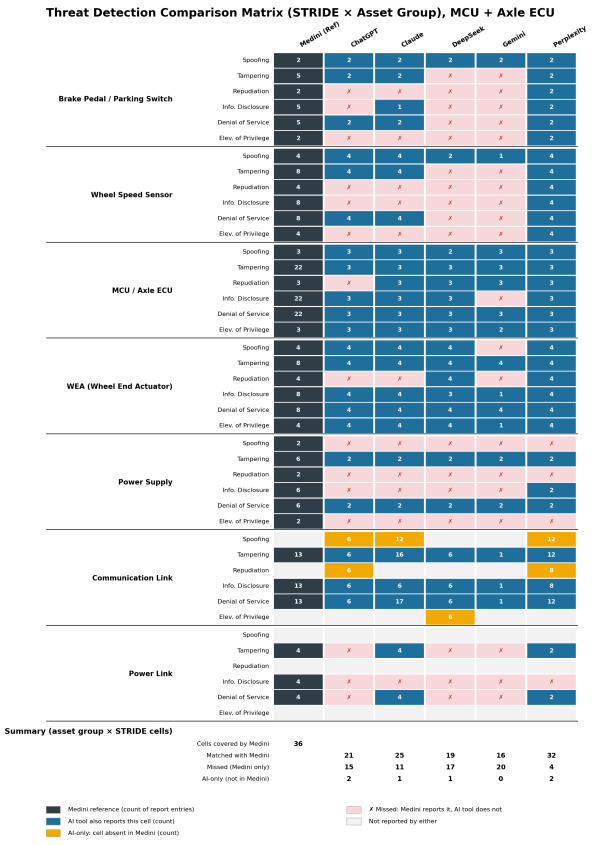


Fig. 18. Threat detection comparison matrix for MCU + Axle ECU.

For MCU + Pedal-direct, Medini reports 36 cells, and the AI models match between 12 and 36 of these cells. DeepSeek matches 36 cells, ChatGPT matches 25, Claude matches 16, Gemini matches 15, and Perplexity matches 12. Medini-only cells range from 0 (DeepSeek) to 24 (Perplexity), and AI-only cells range from 0 (Gemini) to 6 (DeepSeek).

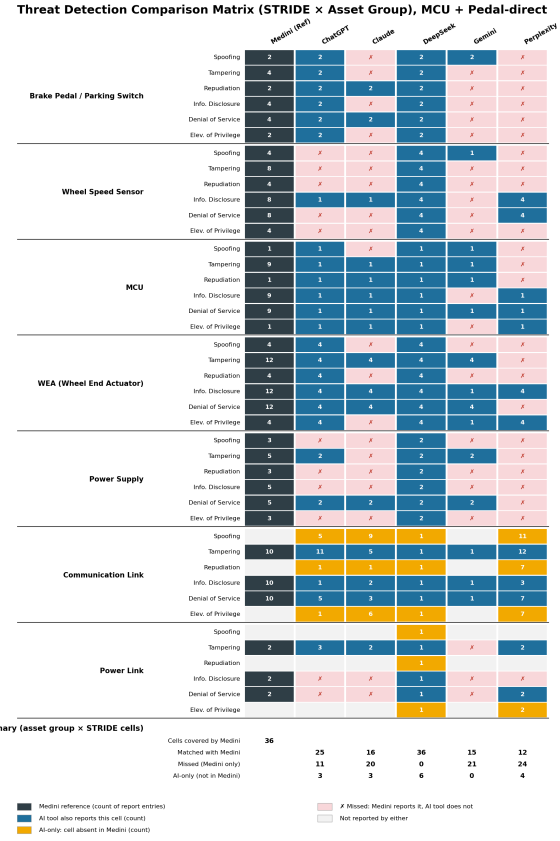


Fig. 19. Threat detection comparison matrix for MCU + Pedal-direct.

For MCU + Auxiliary ECU, Medini reports 36 cells, and the AI models match between 12 and 27 of these cells. DeepSeek matches 27 cells, ChatGPT matches 26, Claude matches 25, Gemini matches 16, and Perplexity matches 12. Medini-only cells range from 9 (DeepSeek) to 24 (Perplexity), and AI-only cells range from 0 (DeepSeek, Gemini) to 4 (Perplexity).

These results indicate that no single AI model consistently outperforms the others across all architectures. DeepSeek performs well on MCU + Pedal-direct and MCU + Auxiliary ECU, but less well on MCU + Axle ECU. Perplexity performs well on MCU + Axle ECU but poorly on MCU + Pedal-direct. ChatGPT and Claude show moderate performance across all three.

E. Tool-Specific Findings

Beyond the quantitative comparison, the analysis reveals distinct tool-specific findings that are not shared across all tools. These findings highlight the complementary strengths of Medini and the AI models.

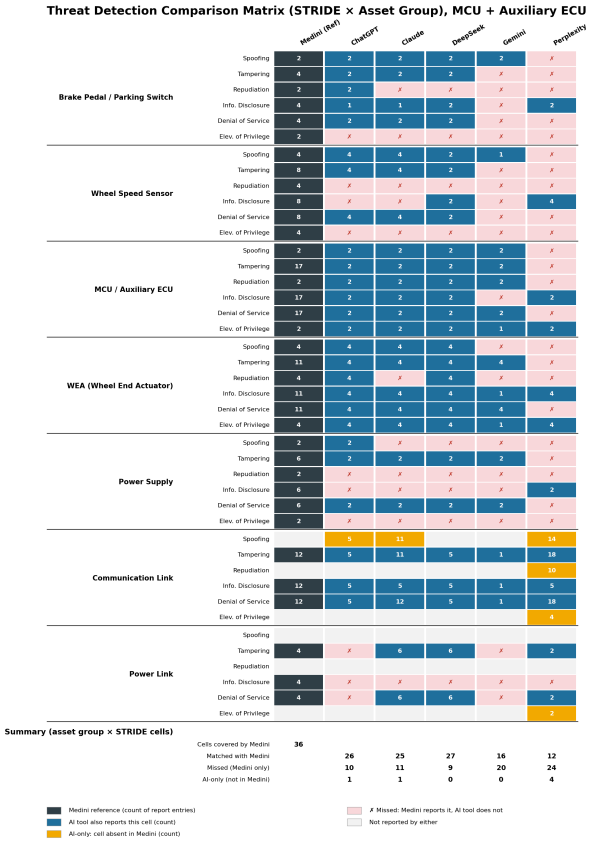


Fig. 20. Threat detection comparison matrix for MCU + Auxiliary ECU.

For **MCU + Axle ECU**, Medini uniquely identifies a compromise of the redundant brake-pedal-to-MCU link, covering Tampering, Denial of Service, and Information Disclosure, with a High rating for DoS and a noted safety impact. The AI models uniquely identify Spoofing on the MCU-to-Axle-ECU links, Elevation of Privilege via the MCU towards Axle ECU links, Denial of Service on the shared communication bus, and Spoofing of actuator command links.

For **MCU + Pedal-direct**, the AI models uniquely identify Denial of Service on the communication bus (a single flood can stop all four actuators), Elevation of Privilege via the bus (a compromised WEA pivoting to gain MCU-level authority), and Spoofing of the bus (without message authentication, any node can impersonate the MCU). Medini uniquely identifies tampering of a single MCU-to-WEA flow (manipulating one wheel's command creates uneven braking force), Denial of Service on the wheel-speed links per corner, and Denial of Service on the power supply to the MCU, modeled as A/B redundancy.

For **MCU + Auxiliary ECU**, Medini uniquely identifies Information Disclosure on the power-supply flows, wheel-speed links, brake pedal wire, and parking-switch wire, plus a Repudiation entry. The AI models uniquely identify Elevation of Privilege through the MCU-to-Auxiliary-ECU link, Spoofing on the MCU-to-Auxiliary-ECU link, and Denial of Service on the shared communication bus.

F. Summary of Findings

Overall, the results suggest four main observations. First, the total number of identified threats increases with architectural complexity, with MCU + Axle ECU receiving the highest number of entries across all tools. Second, AI models tend to report fewer threats than Medini in the Repudiation and Information Disclosure categories, while over-reporting in Tampering and Denial of Service. Third, model performance is architecture-dependent, and no single model consistently matches the Medini reference across all three architectures. Fourth, Medini and the AI models identify complementary threats: Medini tends to capture redundancy-related and fault-tree-style reasoning, while the AI models tend to capture bus-level and link-level attack scenarios. These findings are discussed further in Section V.

V. Discussion

This section interprets the results presented in Section IV, explains the observed differences between Medini and the AI models, and discusses the implications for AI-assisted TARA in automotive cybersecurity.

A. Overall Performance of AI Tools vs. Medini

The results show that no single AI model consistently matches the Medini reference across all three architectures. DeepSeek performs well on MCU + Pedal-direct (36/36 cells) and MCU + Auxiliary ECU (27/36 cells), but less well on MCU + Axle ECU (19/36 cells). Perplexity shows the opposite pattern, performing well on MCU + Axle ECU (32/36 cells) but poorly on MCU + Pedal-direct (12/36 cells). ChatGPT and Claude show moderate performance across all three, while Gemini consistently reports the lowest number of threats.

This variation suggests that AI performance is not solely determined by model capability but also by how well the model's internal representation aligns with the specific architecture. Architectures with fewer components and simpler communication paths (MCU + Pedal-direct) appear to be easier for some models, while architectures with more components and distributed controllers (MCU + Axle ECU) are easier for others. This is consistent with the observation that AI models tend to perform better on architectures whose structure resembles common patterns in their training data.

1) Spoofing Inversion: The results show an inversion in the Spoofing category: Medini reports 43 Spoofing threats across the three architectures, while Claude (57), ChatGPT (50), and Perplexity (50) report more. One possible explanation is that spoofing attacks are relatively straightforward to describe at the component and link level, where AI models operate. For example, "Spoofing of MCU" or "Spoofing of WEA/FL" can be generated directly from the architecture diagram without requiring deeper system knowledge.

By contrast, Medini reports more threats in Tampering, Information Disclosure, and Denial of Service, which may require reasoning about redundancy, fault propagation, and

trust boundaries. These categories are less visible in the architecture diagram and depend on the analyst's understanding of the system's safety goals and fault-tolerant control strategy.

However, since the AI-only findings were not manually validated, it is not possible to determine whether these additional Spoofing threats are genuine or represent over-reporting. It is also possible that AI models use the term "Spoofing" more broadly than Medini, capturing threats that would be classified differently in a human-driven TARA.

B. Why Performance is Architecture-Dependent

The architecture-dependent performance can be explained by three factors. First, the number of components and communication links varies across the three architectures. MCU + Axle ECU has the highest number of components (MCU, two axle ECUs, four WEAs, sensors, power supplies, communication buses), which increases the number of potential attack surfaces. Models that can reason about complex topologies tend to identify more threats in this architecture.

Second, the redundancy structure differs across architectures. MCU + Axle ECU uses an "H"-circuit layout for both power and communication, which introduces same-side redundancy but also same-side vulnerability. MCU + Pedal-direct uses full redundancy for power and an "X"-circuit for communication, which distributes the risk differently. MCU + Auxiliary ECU combines an "X"-circuit for power with an "H"-circuit for communication. These structural differences affect which threats are relevant and which are not.

Third, the abstraction level of the AI models varies. Some models reason at the component level (e.g., individual WEAs, sensors), while others reason at the link level (e.g., communication buses, power lines). This affects which threats are identified and which are missed. Medini, by contrast, operates at the architectural level with explicit fault-tree and redundancy reasoning, which allows it to identify threats that AI models may overlook.

C. Complementary Strengths of Medini and AI

The tool-specific findings in Section IV-E show that Medini and the AI models identify complementary threats. Medini tends to capture:

- Redundancy-related threats (e.g., compromise of the redundant brake-pedal-to-MCU link, A/B power supply redundancy).
- Fault-tree-style reasoning (e.g., Denial of Service on wheel-speed links per corner).
- Information Disclosure on power-supply flows, wheel-speed links, brake pedal wire, and parking-switch wire.

The AI models tend to capture:

- Bus-level threats (e.g., Denial of Service on the shared communication bus, Spoofing of the bus).
- Link-level threats (e.g., Spoofing on MCU-to-Axle-ECU links, Elevation of Privilege via the MCU towards Axle ECU links).
- Attack scenarios that exploit missing authentication (e.g., a compromised WEA pivoting to gain MCU-level authority).

This complementarity suggests that AI models can be used as a *first-pass tool* to identify bus- and link-level threats, while Medini remains necessary for redundancy-related and fault-tree-style reasoning. A combined workflow, in which AI generates a draft TARA and Medini validates and extends it, may provide better coverage than either tool alone.

D. Severity Rating Discrepancies

The severity ratings show notable discrepancies between tools. Gemini and DeepSeek report the highest share of severe-rated threats (79–86% and 54–75%, respectively), while Claude reports the lowest (38–64%). This variation may reflect differences in how each model interprets the safety impact of a threat, or differences in the level of detail provided in the prompt.

It should be noted that DeepSeek and Gemini did not provide native severity ratings in their outputs. For these models, severity values were estimated from the ratings assigned by ChatGPT, Claude, and Perplexity to threats in the same asset group and STRIDE category. This estimation introduces uncertainty, and the severity comparisons involving these two models should be interpreted with caution.

The severity discrepancies also highlight a broader challenge: severity assessment in TARA is inherently subjective and depends on the analyst's judgment. AI models, lacking explicit safety goals and domain knowledge, may assign severity based on surface-level features rather than the actual safety impact. This suggests that severity ratings should always be validated by a human expert before being used in a TARA.

E. Hallucinations and Missed Threats

The AI-only findings (cells not present in the Medini reference set) range from 0 (Gemini) to 6 (DeepSeek for MCU + Pedal-direct). These additional findings may represent either genuine gaps in the Medini analysis or hallucinated threats. Without manual validation, it is not possible to determine which is the case for each finding.

Similarly, the Medini-only findings (cells identified by Medini but missed by the AI) range from 0 (DeepSeek for MCU + Pedal-direct) to 24 (Perplexity for MCU + Pedal-direct). These missed threats are particularly important because they represent potential blind spots in the AI analysis.

The Repudiation category is the least covered across all AI models, with Gemini reporting only 6 entries. This suggests that AI models struggle with threats that involve denying an action or repudiating a message, possibly because these threats require reasoning about logging and accountability mechanisms that are not visible in the architecture diagram.

F. Implications for AI-Assisted TARA

The results have three main implications for the use of AI in TARA:

1) **AI as a complement, not a replacement.** AI models can accelerate threat identification and provide a first-pass analysis, but they cannot replace expert-driven TARA. Medini remains necessary for redundancy-related reasoning, fault-tree analysis, and severity validation.
2) **Architecture-aware prompting.** The architecture-dependent performance suggests that prompts should be tailored to the specific architecture, highlighting the components, communication links, and redundancy structure that are most relevant for the analysis.
3) **Human validation is essential.** The AI-only findings and missed threats show that AI outputs must be validated by a human expert before being integrated into a TARA. This is consistent with the ISO/SAE 21434 requirement that TARA is a human-driven process.

G. Limitations

This study has several limitations. First, the analysis is based on a single prompt template, and each prompt was executed only once; different prompts and repeated runs may yield different results. Second, the AI models were evaluated on architecture diagrams only, without access to the underlying system specification or safety goals. Third, the study focuses on three EMB architectures derived from a single reference paper; other architectures may yield different results. Fourth, the Medini-based reference TARA was performed by a human analyst, and its completeness depends on the analyst's expertise and the available system information. Fifth, the AI models were evaluated at a single point in time; model updates may change their performance. Future work should address these limitations by testing multiple prompts, repeating the analysis across multiple runs, providing richer context to the AI models, and extending the analysis to other architectures and vehicle systems.

VI. Conclusion

This paper presented a comparative TARA methodology for EMB systems, in which STRIDE-based threat analysis was performed both by a human analyst using ANSYS Medini Analyze and by five AI models (Perplexity Sonar, Gemini 3.8 Flash, DeepSeek-V3, Claude Sonnet 5, and GPT-5.6 Luna). Three candidate EMB architectures—MCU + Axle ECU, MCU + Pedal-direct, and MCU + Auxiliary ECU—were analyzed using the same prompt and the same architecture image across all tools. The results were compared in terms of cell coverage, STRIDE category distribution, severity ratings, and tool-specific findings.

The main findings are as follows:

- AI performance is architecture-dependent: no single model consistently matches the Medini reference across all three architectures. DeepSeek performs well on MCU + Pedal-direct and MCU + Auxiliary ECU, while Perplexity performs well on MCU + Axle ECU.
- Medini and the AI models identify complementary threats. Medini captures redundancy-related and fault-tree-style threats, while the AI models capture bus-level and link-level attack scenarios.
- AI models tend to under-report Repudiation and Information Disclosure threats, and over-report Tampering and Denial of Service threats compared to Medini.
- Severity ratings vary notably across tools, and two models (DeepSeek and Gemini) did not provide native severity ratings, requiring estimation.
- AI-only findings and missed threats indicate that AI outputs must be validated by a human expert before being integrated into a TARA.

These findings suggest that AI models can serve as a complement to expert-driven TARA by accelerating threat identification and providing a first-pass analysis, but they cannot replace Medini-based analysis. A combined workflow, in which AI generates a draft TARA and a human analyst validates and extends it in Medini, may provide better coverage than either tool alone.

Future work will address the limitations of this study by testing multiple prompt templates, providing richer context to the AI models, validating the AI-only findings with domain experts, and extending the analysis to other EMB architectures and vehicle systems. The methodology may also be adapted to other safety-critical automotive systems where both functional safety and cybersecurity must be considered.

Appendix

You are an automotive cybersecurity analyst performing a threat analysis using the STRIDE methodology (Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, Elevation of Privilege).

SYSTEM

The system is an electro-mechanical brake (EMB) system for passenger vehicles. There are no mechanical or hydraulic connections between the brake pedal and the wheels; all braking commands are transmitted electrically to four wheel-end actuators (WEAs), each containing an actuator control unit, a servo motor and a brake caliper.
The attached figure shows the architecture to analyse:
[PASTE ONE ARCHITECTURE DESCRIPTION HERE]
The system has three safety goals:

- SG1: Avoid loss of or insufficient braking (deceleration below 5.15 m/s^2)
- SG2: Avoid unintended braking (deceleration above 2.44 m/s^2)
- SG3: Avoid braking deviation (lateral displacement above 1.20 m)

TASK

Identify as many STRIDE threats as you can for this architecture. Consider every component shown in the figure (controllers, WEAs, sensors, brake pedal, parking switch, power supplies) and every communication link between them.

RULES

- One threat per row. Do not merge several threats into one row.
- Cover all six STRIDE categories where applicable.
- Use the component names exactly as labelled in the figure (e.g. WEA/FL, MCU, Axle ECU/R).

- Focus on threats that result from this architecture's specific structure (controllers, power and bus layout).
- Base your analysis only on the information provided.
- Do not repeat the same threat with different wording.

OUTPUT FORMAT

Fill in the table below. Column rules:

- ID: T001, T002, T003, ...
- Name: "¡STRIDE category¿ of ¡asset¿", e.g. "Spoofing of MCU"
- Assets: the component(s) or link(s) affected, as labelled in the figure
- To Be Assessed: true if the threat could violate SG1, SG2 or SG3, otherwise false
- Category: one of Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, Elevation of Privilege
- Severity: Severe, Major, Moderate or Negligible, based on the worst effect on the safety goals; "-" if it cannot be judged

ID	Name	Assets	To Be Assessed	Category
T001	Information Disclosure of WEA/FL	WEA/FL	true	Information Disclosure

Output only the completed table (continue from the example row), followed by the total number of threats per STRIDE category.

REFERENCES

REFERENCES

[1] *ISO/SAE 21434: Road Vehicles—Cybersecurity Engineering*, International Organization for Standardization, Geneva, Switzerland, 2021.

[2] J. Peng, T. Wu, L. Chu, J. Rong, X. Yang, and Y. Meng, "Electro-Mechanical Brake System Architectural Design and Analysis Based on Functional Safety of Vehicles," *Actuators*, vol. 13, no. 9, p. 346, 2024.

[3] Z. Zhu and L. Li, "Architectural Design and Analysis of Electro-Mechanical-Brake System in View of Functional Safety Concept," *IEEE Trans. Intell. Veh.*, vol. 10, no. 8, pp. 4330–4342, Aug. 2025.

[4] H. N. Faisal, F. M. Mohammed, and J. A. Mohammed, "Design and Implementation of an Electromechanical Brake System," *Eng. Technol. J.*, vol. 40, no. 1, pp. 1–9, 2022.

[5] S. S. Gunjate and S. A. Khot, "A Systematic Review of Emergency Braking Assistant System to Avoid Accidents Using Pulse Width Modulation and Fuzzy Logic Control Integrated with Antilock Braking," *Int. J. Automot. Mech. Eng.*, vol. 20, no. 2, pp. 10457–10479, 2023.

[6] X. Zhao, L. Xiong, G. Zhuo, W. Tian, J. Li, Q. Shu, X. Zhao, and G. Xu, "A Review of One-Box Electro-Hydraulic Braking System: Architecture, Control, and Application," *Sustainability*, vol. 16, no. 3, p. 1049, 2024.

[7] P. Park, H. Choi, S. Lee, S. Jeong, and H. Lee, "Development of a novel electro-mechanical brake motor thermal management system for nonuniform heating under extreme thermal conditions," *Energy Convers. Manag.*, vol. 325, p. 119406, 2025.

[8] R. Ribeiro, C. Gonçalves, and N. Pombo, "Elevating Traffic Safety: Insights into Autonomous Emergency Braking Systems in Varied Highway Environments," in *Proc. IEEE 22nd Int. Conf. Softw. Archit. Companion (ICSA-C)*, 2025.

[9] U. Della Monica, K.-A. Munjal, M. P. Tamas, B. Boi, C. Esposito, and R. Khondoker, "TARA Analysis of an Autonomous Emergency Braking (AEB) System," *Appl. Sci.*, vol. 15, no. 3, p. 1400, 2025.

[10] F. Wang, D. Chen, Y. Cai, X. Xu, and Y.-Q. Ni, "Nonlinear Clamping Force Estimation and Robust Control of an Electro-Mechanical Brake System for Electric Vehicles Based on PWA-H," *IEEE/ASME Transactions on Mechatronics*, 2024.

[11] B. Shi, L. Xiong, and Z. Yu, "Pressure Estimation Based on Vehicle Dynamics Considering the Evolution of the Brake Linings' Coefficient of Friction," *Actuators*, vol. 10, p. 76, 2021.

[12] X. Wu, R. He, H. Ge, and M. Chen, "Electronic mechanical braking system executive mechanism design, calculation, and modeling based on dynamic control," *Frontiers in Mechanical Engineering*, 2024.

[13] S. Schrade, X. Nowak, A. Verhagen, and D. Schramm, "Short Review of EMB Systems Related to Safety Concepts,"

[14] J. A. Khan, M. A. Khan, N. Saeed, P.-L. Cayrel, and C. Hahn, "Intrusion Detection Systems for In-Vehicle Networks: Protocols, Applications, and Challenges," *IEEE Access*, 2025.

[15] Y. F. Fu, X. H. Hu, W. R. Wang, and Z. Ge, "Simulation and experimental study of a new electromechanical brake with automatic wear adjustment," *International Journal of Automotive Technology*, vol. 21, no. 1, pp. 227–238, 2020.